TEAM INITIATORS · DISTRICT IQ REVIEW · 18 OCT 2026, 4:45 PM

Study guide

The whole system, from data collection to the website

Read this top to bottom once. Each step says what happens, which algorithm does it, and the one line to say if asked.

The big picture

Layer	What it is	Where in the code
Collectors	One program per source; each keeps its own fetch method	`imd_weather_collector/`, `cpcb_air_quality_collector/`, `cfm_dss_collector/`, `chennai_news_pipeline/`
Simulators	Realistic department records: grievances, police, PWD, hospitals	`police_dataset_generator/`, `pwd_dataset_generator/`, `chennai_hospital_data/`, the portal's grievance generator
Pipeline	Cleans, locates, links and analyses everything into one store	`district_intel/` (`dintel/*.py`)
Scheduler	Hourly job on GitHub Actions, triggered by cron-job.org; the PC sends CPCB and CFM	separate repo `farmwisenew-dataload`, `district_intel/scripts/run_intel.bat`
Storage	AWS S3 (each build as compressed tables plus a manifest), DynamoDB (accounts, complaints, decisions)	`aws/` (kept on the team PC)
Website	Next.js app: citizen portal, Collector console, officer consoles, Ask District IQ, Data Studio	`chennai-grievance-portal-main/`

Data collection, feed by feed

IMD weather (real)

IMD city-weather JSON for 4 stations: 24-hour rain to 8:30 AM, temperature, humidity, 7-day forecast. District warnings from IMD's GIS warning map (WFS). Endpoints behind a login or IP whitelist are logged, never bypassed.

CPCB air quality (real)

Playwright opens CPCB's public city map and reads each station's AQI popup. data.gov.in needs a key we don't have; history is behind a captcha, so missing days are reported, not invented. Runs on the team PC (CPCB blocks cloud servers).

Chennai Flood Monitor, CFM-DSS (real)

TN WRD's flood DSS API: 25 river and canal gauges with warning and danger levels, reservoir records, alerts and bulletins; Adyar, Cooum, Kosasthalayar and Kovalam basins. Also from the PC since 14 Oct.

News (real)

RSS and sitemaps: The Hindu, Times of India, DT Next, New Indian Express, Dinamani, Dinamalar, plus Google News search. Full text from publisher pages where robots.txt allows. A Tamil-aware gazetteer finds places.

Lakes and prices (real, read by the website)

Chennai Metro Water (CMWSSB) lake levels; AGMARKNET and Uzhavar Sandhai vegetable and fruit prices; nearby-district news (Tiruvallur, Kancheepuram, Chengalpattu).

Simulated departments

Grievances (about 16,100), police (about 12,300), PWD (about 2,100) and hospital alert episodes, 180 days. One shared rain calendar (`dintel/sync.py`) so all sources react to the same rain days. Every row has `is_synthetic = 1` .

The pipeline, step by step

1. **Load.** One loader per source turns every record into a row of one `events` table (70 columns: time, text, category, place, impact, severity, status, provenance). Mobile numbers become a keyed hash, so personal data is never copied.

2 **Classify.** Structured sources use crosswalk tables to our ~31 categories. Free text uses a character n-gram classifier (scikit-learn), which handles English, Tamil and Tanglish. News is labelled by an LLM (Groq), and a local SetFit model (multilingual-e5-base) trained on those labels covers articles the LLM hasn't reached. A trained incident filter decides which articles are real incidents and which are notices, politics or court news.

3 **Locate.** Point-in-polygon on the 200 GCC ward boundaries; a point within 500 m outside a ward is snapped (and flagged). Ward → zone, and ward → taluk by majority vote. News places come from the gazetteer, which picks *where it happened*, not every place mentioned.

4 **Shared world (overlay).** The simulators were written separately, so they disagreed about rain days. The overlay builds one world rain calendar and adds the reports a source would have produced on those days. It also plants multi-source events, which give us an answer key to measure the linker. Real sources are never changed.

5 **Duplicates within a source.** Compare only plausible pairs (blocking), apply the portal's rule for grievances plus officers' own "duplicate of" decisions, then **union-find** joins them into groups. Measured with **B-cubed F1 ≈ 0.995**.

6 **News stories across languages. bge-m3** embeddings put Tamil and English in one vector space to find candidate pairs; the **bge-reranker-v2-m3** cross-encoder reads each pair together; a small model trained on our labelled pairs decides; an LLM checks unsure pairs; and we stop chaining (A like B, B like C does not pull in C). About 9,500 story groups became 7,100; about 2,800 merges joined Tamil and English reports.

7 **Link across sources into incidents.** Blocking by category group, a **KD-tree** on distance and a time window per category → a **logistic regression** scores each pair on 9 features: distance, time gap, word similarity, meaning similarity (multilingual-e5), shared named things, same category, same ward, coarse location, involves news → thresholds learned from the data (merge at 0.80, review band from 0.55) → union-find. A news report links to a department record only if both have the same category. **F1 ≈ 0.83** on the held-out half (precision 0.97, recall 0.72).

8 **Incidents: severity, deadline, priority.** Lead record, status and a deadline per category (an SLA). **Severity** is a rule score from 0 to 100 (≥70 Severe, ≥50 High, ≥30 Medium, else Low), from facts only. **Priority** = severity + weighted signals, each with its reason (table below). Priority is computed when it is read, so it never goes stale.

9 **Analytics.** Poisson baselines with weekday and rain factors; one-day spikes plus **CUSUM** slow rises; all ~3,900 tests under one **Benjamini-Hochberg** FDR at 5%. **DBSCAN** hotspots, **Getis-Ord Gi*** on wards, **EWMA** and z-scores for trends, KPIs per period and zone, and **PSI** for category drift.

10 **Agents. Data Steward** (feed health, data contract checks, quarantine, drift), **Action Planner** (playbook tasks per category), **Gap Finder** (in the news, no department record), **Linker** (pairs in the review band), **Watchdog** (alerts), **Briefing** (writes the daily, weekly, monthly and quarterly briefings; a numeric verifier rejects any number not in the data: 0 unverified today).

11 **Store and publish.** SQLite store (33 tables) → `export_intel.py` uploads each changed table to S3 as gzip JSON, named by its hash, and then a manifest. Unchanged tables are skipped.

12 **Website reads it.** On start and every few minutes, the site downloads the manifest and tables into an in-memory SQLite database (Node's built-in `node:sqlite`). Its queries were written for MySQL, so a small dialect layer translates them. Build hooks add headlines, incident clusters and lake alerts. Writes (complaints, decisions) go to DynamoDB through API Gateway and a Lambda.

Priority signals and weights

Signal	Points	Meaning
Past deadline, twice over	`+20`	open for more than twice its category's SLA
Past deadline	`+10`	open beyond its SLA
Serious and unverified	`+8`	severe or high, no officer has confirmed it on the ground
Growing	`+8`	3 or more new reports in the last 24 hours
More than one source	$5 \times \log_2(\text{sources})$	e.g. 2 sources = +5, 4 sources = +10
Vulnerable people	`+5`	the record names an affected vulnerable group
Several outlets	`+5`	2 or more news outlets
Only in the media	`+5`	news reports it, no department record
Rain	`+5`	linked to a rain event
Recurring	`+5`	3 or more times at the same place in 90 days
Citizen complaints	$3 \times \log_2(1 + \text{complaints})$	applies from 2 complaints; grows slowly so many complaints can't swamp severity

The deadline and growth signals apply only while the incident is open; "serious and unverified" applies only to Severe or High.

Learning: each Collector decision (verify, reopen, escalate) is saved with the incident's facts. After 30 decisions, with at least 5 on each side, a logistic model refits the weights with a Gaussian prior around these hand-set values. The learned weights are used only if they rank held-out decisions at least as well. Today: 6 decisions, still collecting.

Ask District IQ, step by step

1. **Code guard.** Rejects unsafe or off-topic input; fixes spelling; detects Tamil.

2. **Router** (a fast model, or rules if no AI is available) decides the kind of question: a count, a list, a chart, an explanation, a console action, or a chart edit.

3. **Tools or query plan.** Either one of the console's own functions, or a planner writes a structured plan that a **compiler** turns into **read-only SQL**. The model never writes raw SQL.

4. **Search** for open questions: **LanceDB** (embedded vector database) with **multilingual-e5-base** vectors, plus BM25 keyword search, merged by **reciprocal-rank fusion**. It retrieves records; counts still come from SQL.

5. **Facts → composer.** Code builds the facts; the composer model writes the answer in English or Tamil.

6. **Number verifier.** Every number in the text must exist in the facts. If not, it regenerates once, then falls back to a template. A chart checker validates the chart type.

7. At most three model calls per question. If the AI is busy or missing, the answer still comes from the tools and says "without AI". Officers' answers are locked to their department.

Tech stack

Area	Technology

Website	Next.js 14, React 18, TypeScript, Tailwind CSS, Leaflet (maps), ECharts (charts), Motion and anime.js (animation), jsPDF (PDF export), SheetJS (Excel), Zod (validation)
Website data	In-memory SQLite (`node:sqlite`) loaded from S3; durable writes to DynamoDB; MySQL still supported (`DATA_BACKEND=mysql`)
Security	bcrypt passwords, signed JWT session cookies (jose), role guards, login rate limits, OTP, encrypted ID numbers, SSRF guard on fetched links, audit log, email sandbox with an allowlist
Pipeline	Python 3.11, pandas, NumPy, scikit-learn, SciPy, sentence-transformers, SetFit, Playwright, truststore
Models	multilingual-e5-small (linking), multilingual-e5-base (news classifier, chatbot search), bge-m3 and bge-reranker-v2-m3 (cross-language stories)
AI services	Amazon Bedrock (Nova Lite / Nova Micro); Gemini and Groq as fallbacks; Transformers.js runs the embedding model inside the website
AWS	S3, DynamoDB, Lambda, API Gateway, CloudWatch Logs, Bedrock; region ap-south-1 (Mumbai)
Automation	GitHub Actions (hourly build), cron-job.org (trigger), Windows Task Scheduler on the PC (CPCB and CFM)
Testing	Vitest (about 570 web tests), pytest (about 100 pipeline and 88 news tests), a golden set of 75 chatbot questions

Glossary: one line each

Term	Say this
Union-find	A fast way to merge pairs into groups: if A=B and B=C, all three become one group.
Blocking	Only compare records that could plausibly match (same category group, nearby, close in time) instead of every pair.
KD-tree	A spatial index that finds nearby points quickly.

Logistic regression	Turns the pair features into a probability that two reports are the same event.
Embedding	A vector that represents the meaning of a text; similar meanings sit close together, even across Tamil and English.
Cross-encoder (reranker)	Reads two texts together and scores how related they are; slower but more accurate than comparing vectors.
SetFit	Fine-tunes a sentence-embedding model with few labels to make a classifier.
Poisson baseline	The expected count per day for a category in a zone, from its own history.
CUSUM	Cumulative sum test that detects a slow, steady rise rather than a one-day spike.
Benjamini-Hochberg (FDR)	When you run thousands of tests, it controls the share of false alarms among the alerts you keep (5%).
DBSCAN	Density clustering: finds tight groups of points (hotspots) without fixing the number of clusters.
Getis-Ord Gi*	Says whether a ward and its neighbours together have unusually high counts: a statistical hotspot.
EWMA	An exponentially weighted moving average that smooths a trend.
PSI	Population stability index: measures how much the category mix has drifted.
B-cubed F1	A clustering score: for each record, how correct its group is.
Precision / recall / F1	Precision: of what we flagged, how much was right. Recall: of what was really there, how much we found. F1 balances the two.
Reciprocal-rank fusion	Merges two ranked result lists (meaning search and keyword search) into one.
Point-in-polygon	Tests which ward boundary contains a latitude and longitude.